

Práctica 2: Resolución del problema del agente viajero con algoritmos genéticos

Brian Miguel Escalona Maldonado, brian.escalona@alumnos.uacm.edu.mx

Resumen—debería ser considerado como una mini versión del trabajo. El resumen tiene típicamente un único párrafo y usualmente es de 150 a 250 palabras, debe "permitir a los lectores identificar el contenido básico del documento rápido y fielmente, con el fin de determinar la relevancia del mismo para sus intereses y, por tanto, para decidir si necesitan leer el documento en su totalidad".

I. INTRODUCCIÓN

En esta práctica, se desea implementar un algoritmo evolutivo para resolver el problema del agente viajero que visita n ciudades de acuerdo con ciertas restricciones. El problema del **Agente Viajero (Travelling Salesman Problem o TSP)** consiste en encontrar la ruta más corta que recorra un conjunto de ciudades, pasando por cada una de ellas exactamente una vez y regresando al punto de origen.

Se tienen las siguientes 20 ciudades por visitar:

```
1 ciudades = [
2   {"nombre": "Ciudad de Mexico", "coords":
3     ↪ (19.4326, -99.1332)},
4   {"nombre": "Nueva York", "coords":
5     ↪ (40.7128, -74.0060)},
6   {"nombre": "Londres", "coords": (51.5074,
7     ↪ -0.1278)},
8   {"nombre": "Tokio", "coords": (35.6762,
9     ↪ 139.6503)},
10  {"nombre": "Sidney", "coords": (-33.8688,
11    ↪ 151.2093)},
12  {"nombre": "Paris", "coords": (48.8566,
13    ↪ 2.3522)},
14  {"nombre": "Berlin", "coords": (52.5200,
15    ↪ 13.4050)},
16  {"nombre": "Roma", "coords": (41.9028,
17    ↪ 12.4964)},
18  {"nombre": "Sao Paulo", "coords":
19    ↪ (-23.5505, -46.6333)},
20  {"nombre": "Buenos Aires", "coords":
21    ↪ (-34.6037, -58.3816)},
22  {"nombre": "El Cairo", "coords": (30.0444,
23    ↪ 31.2357)},
24  {"nombre": "Moscu", "coords": (55.7558,
25    ↪ 37.6173)},
26  {"nombre": "Beijing", "coords": (39.9042,
27    ↪ 116.4074)},
28  {"nombre": "Delhi", "coords": (28.6139,
29    ↪ 77.2090)},
30  {"nombre": "Johannesburgo", "coords":
31    ↪ (-26.2041, 28.0473)},
```

```
17 {"nombre": "Toronto", "coords": (43.6532,
18   ↪ -79.3832)},
19 {"nombre": "Dubai", "coords": (25.2769,
20   ↪ 55.2962)},
21 {"nombre": "Singapur", "coords": (1.3521,
22   ↪ 103.8198)},
23 {"nombre": "Estocolmo", "coords":
24   ↪ (59.3293, 18.0686)},
25 {"nombre": "Auckland", "coords":
26   ↪ (-36.8485, 174.7633)}
27 ]
```

Esta lista con las 30 ciudades contiene dos campos que agrupan los datos de la siguiente manera:

- **Primer campo:** Nombre de la ciudad
- **Segundo campo:** Coordenadas de la ciudad (latitud y longitud)

Para esta práctica se considero una representación en la que cada gen (o sea, cada elemento de la lista) representa a una ciudad, identificada por un número único, en este caso del 0 al 19 para nuestras 20 ciudades. En esta representación se tiene una restricción y esta es que todos los individuos deben iniciar con la **Ciudad de México**, representada por el **número 0**. Ya que tenemos la representación de las ciudades, generamos un individuo aleatorio; dicho individuo es el siguiente:

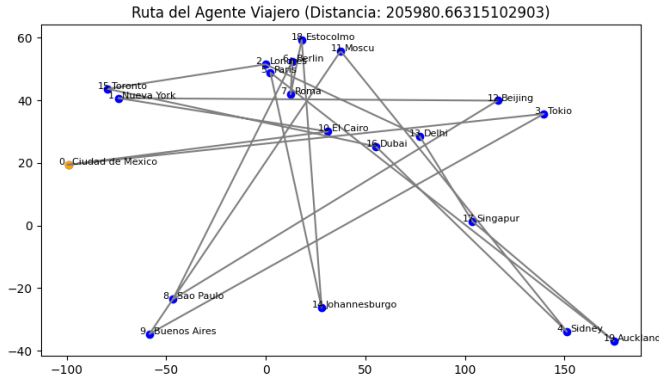
```
1 [ 0, 10, 1, 12, 8, 6, 7, 18, 14, 5, 19,
   ↪ 17, 13, 2, 15, 16, 4, 11, 9, 3]
```

Este individuo representa un recorrido particular por todas las ciudades. Presentando el nombre de las ciudades, el individuo contiene el siguiente recorrido:

```
1 'Ciudad de Mexico', 'El Cairo', 'Nueva
   ↪ York', 'Beijing', 'Sao Paulo',
2   ↪ 'Berlin', 'Roma', 'Estocolmo',
3   ↪ 'Johannesburgo', 'Paris', 'Auckland',
4   ↪ 'Singapur', 'Delhi', 'Londres',
5   ↪ 'Toronto', 'Dubai', 'Sidney', 'Moscu',
6   ↪ 'Buenos Aires', 'Tokio'
```

El recorrido, de manera gráfica, es el siguiente:

Figura 1. Recorrido del individuo de prueba



Como podemos ver en la *Figura: 1* se calcula la distancia del recorrido. Este recorrido tiene un total de 205,980km. Lo que se busca es minimizar este recorrido haciendo uso de algoritmos evolutivos.

II. DESARROLLO

Para el desarrollo de esta práctica se resolverá el problema del agente viajero. En la *Tabla: I* se muestran los parámetros que se utilizarán para evaluar el desempeño de nuestro algoritmo, en esta ocasión para evitar la explosión de combinaciones se realizó un proceso de prueba por fases.

Tabla I
PARÁMETROS DEL ALGORITMO GENÉTICO

Parámetro	Valor
Representación	Permutaciones
Recombinación	PMX; Cruza de bordes; Cruza de órdenes; Cruza de ciclos
Probabilidad de recombinación	100 %
Mutación	Intercambio; Inserción; Desordenada; Inversión
Probabilidad de mutación (<i>tasa_mutacion</i>)	5 %, 10 %, 20 %
Selección de padres	Torneo: 2 mejores de 5
Selección de supervivencia	Con y sin elitismo
Tamaño de la población	50, 100, 200
Número de descendientes	2
Inicialización de la población	Permutaciones con ciudad inicial fija
Función de aptitud	Minimizar la distancia
Condición de terminación	200 generaciones

Las pruebas en tres fases se componen de la experimentación de un único parámetro para evaluar cuál es la mejor selección entre dichos operadores. Dichas fases fueron las siguientes:

II-A. Fase 1. Operadores de cruce

En esta sección, fijamos un único operador de mutación (**Mutación por intercambio**); se fijó el tamaño de la población a 100 individuos y, por último, se fijó la tasa de mutación a un valor del 5 %. Esto con el fin de evaluar únicamente los distintos operadores de cruce:

- PMX
- Cruza de bordes

- Cruza de órdenes
- Cruza de ciclos

Después de realizar los experimentos utilizando un servidor externo y **Temux**, se ejecutaron todas las combinaciones con un máximo de 200 iteraciones para obtener los mejores individuos de esta fase. La *Tabla: II* contiene los individuos con la mejor aptitud; en este caso, aquellos individuos con la aptitud más baja son los que se consideran mejores, ya que lo que se busca optimizar es la distancia entre las ciudades.

Tabla II
RESULTADOS DE COMBINACIONES DE LA FASE 1

Cruza	Mutación	Tasa	Elitismo	Fitness	Diversidad
Bordes	Intercambio	0.05	Sí	69906.16	0.0239
PMX	Intercambio	0.05	No	73603.59	0.0134
Bordes	Intercambio	0.05	No	74209.41	0.0139
Bordes	Intercambio	0.05	Sí	74412.20	0.0305
Bordes	Intercambio	0.05	No	74901.22	0.0192

Como podemos ver en la *Figura: 2* el recorrido es casi de manera "lineal" o "natural", por decirlo de alguna manera. En la *Figura: 3* podemos observar la evolución de la aptitud a través de las 200 generaciones; podemos ver cómo la aptitud máxima oscila de manera bastante agresiva; sin embargo, la aptitud mínima (**la que es relevante para este análisis**) rápidamente se acerca a valores muy bajos. La aptitud media se encuentra muy cercana a la aptitud mínima, lo que significa que los individuos no eran tan lejanos entre sí. Esto se ve de mejor manera con la *Figura: 4*. Conforme la diversidad va disminuyendo, la aptitud promedio se va acercando a la aptitud mínima. De esta manera, podemos notar cómo afecta la diversidad a nuestro algoritmo. En esta fase, las configuraciones tienen una evolución de diversidad y una evolución de aptitud parecidas; lo único en lo que difieren es en el recorrido, que es ligeramente más costoso. Esto se debe a que solo probamos el método de cruce.

Figura 2. Recorrido del mejor individuo de la fase 1

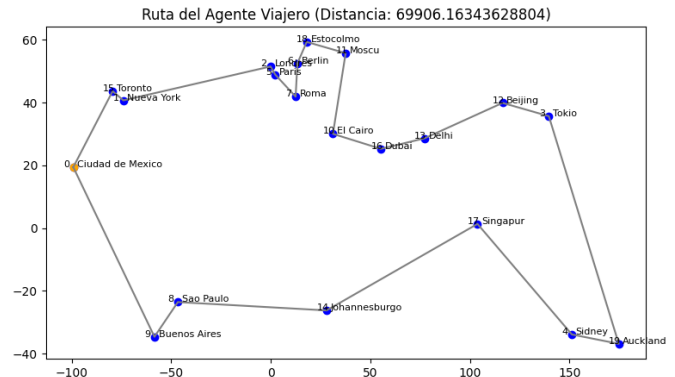


Figura 3. Evolución de la aptitud

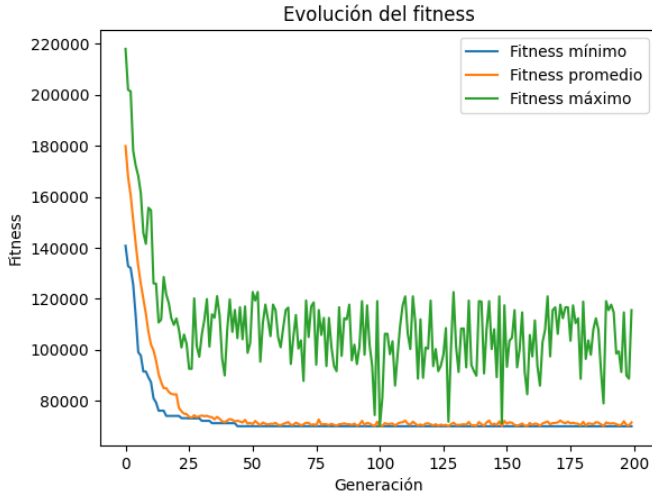
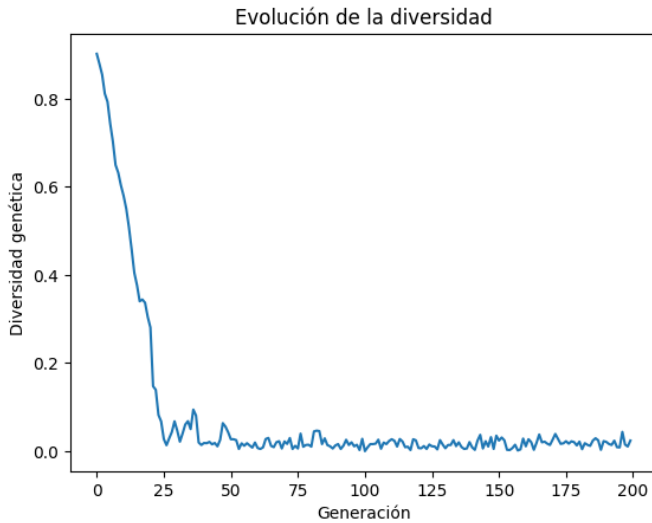


Figura 4. Evolución de la diversidad



Como podemos observar en la *Tabla: II* el operador que más se repite es el **cruce de bordes**, por lo cual será este operador el que utilizaremos en la siguiente fase, en la que compararemos cómo afectan los operadores de mutación a nuestro algoritmo genético. Como pudimos observar en esta etapa, las diferencias son mínimas y solamente difieren en como evolucionaron a través de las épocas.

II-B. Fase 2: Operadores de mutación

Con el resultado de la fase anterior, obtenemos el mejor operador de cruce. Este operador fue el **cruce de bordes**, ya que fue el que mejor resultado tuvo en la fase anterior; por lo tanto fijaremos dicho operador para realizar nuevamente los experimentos. Fijamos el tamaño de la población a 100 individuos; y una tasa de mutación de 0,05%. Estos parámetros fijos nos ayudarán a evaluar los operadores de mutación, los cuales son:

- Intercambio
- Inserción

- Desordenada
- Inversión

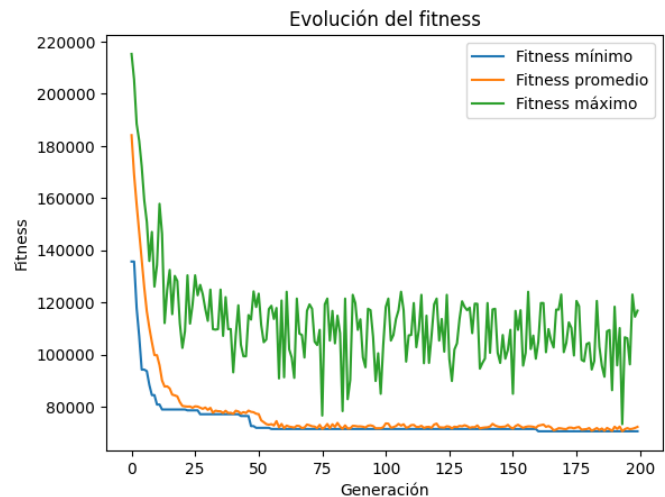
Después de realizar nuevamente los experimentos, se obtuvieron resultados similares a los presentados anteriormente. La primera configuración de la *Tabla: III* es igual a la primera configuración de la *Tabla: II*. Sin embargo, la aptitud es un poco mayor; esto se debe a la inicialización estocástica del algoritmo. Esta pequeña diferencia es mínima en comparación con otros individuos. Como podemos observar en la *Figura: 2* y en la *Figura: 7* el recorrido de ambas configuraciones es muy similar; es por eso que la diferencia es de apenas 475,33km. Esto es únicamente el cambio de una ciudad en el recorrido, por lo que ambas soluciones son óptimas para nuestro problema del agente viajero.

Tabla III
RESULTADOS DE COMBINACIONES DE LA FASE 2

Cruza	Mutacion	Tasa	Elitismo	Fitness	Diversidad
Bordes	Intercambio	0.05	Si	70381.49	0.0339
Bordes	Intercambio	0.05	Si	70513.23	0.0149
Bordes	Intercambio	0.05	Si	70613.10	0.0286
Bordes	Intercambio	0.05	No	70929.37	0.0089
Bordes	Intercambio	0.05	Si	70933.43	0.0086

Como podemos observar en las *Figuras: 5, 6* podemos ver la evolución de la aptitud y la evolución de la diversidad a través de las 200 épocas. Podemos observar que son parecidas a las anteriores; esto significa que nuestras configuraciones generan individuos parecidos.

Figura 5. Evolución de la aptitud



ya tenemos dos configuraciones con individuos muy buenos. Este es el propósito principal de la **Computación Evolutiva**: obtener un individuo que sea suficientemente bueno para resolver un problema en específico, más no el mejor de todo el campo de búsqueda. De esta manera, podemos decir que todos los individuos devueltos por las configuraciones presentadas en la *Tabla: IV* ya son buenos individuos de manera individual y cualquiera puede resolver el problema de una buena manera.

Tabla IV
RESULTADOS DE COMBINACIONES DE LA FASE 3

Cruza	Mutacion	Tasa	Poblacion	Elitismo	Fitness	Divers
Bordes	Intercambio	0.20	200	No	69906.16	0.09
Bordes	Intercambio	0.20	100	No	70929.37	0.06
Bordes	Intercambio	0.05	100	Si	70956.19	0.02
Bordes	Intercambio	0.05	100	Si	71633.43	0.01
Bordes	Intercambio	0.20	200	Si	71810.43	0.0571

Podemos observar que las configuraciones no dependen tanto de los parámetros probados en esta fase, ya que, a pesar de tener más opciones en la *Tabla: IV*, podemos ver que un tamaño de población de 100 individuos es más que suficiente para algunas configuraciones y, de igual manera, la tasa de mutación de 0,20 no tiene una mejora significativa frente a la tasa más baja. Con esto, podemos decir que los valores que parecen no funcionar son la población de 50 individuos y la tasa de mutación de 0,10. Esto probablemente se deba a que una población de 50 individuos es demasiado baja para un problema como este, ya que con una mayor cantidad de población el algoritmo puede probar y explotar más la región de búsqueda, cosa que no ocurre con esta población más pequeña. Mientras que la tasa de mutación de 0,10 simplemente no funciona tan bien como las demás, ya que dentro de las listas de todas las configuraciones se encuentra cerca del lugar 20, lo que nos demuestra que para este problema una tasa de mutación muy pequeña como 0,05 o algo más alta 0,20 son las ideales.

En la *Figura: 10* podemos observar como evoluciona la aptitud a través de las 200 generaciones, podemos observar que a pesar de no contar con elitismo igualmente se mantienen los individuos con buena aptitud esto es importante ya que no se pierden buenos individuos, esto es importante para obtener buenos resultados; la evolución de la diversidad mostrada en la *Figura: 11* nos muestra una mayor diversidad comparada con los mostrados por configuraciones con el elitismo activado, esto se debe a que con cada generación se permiten explorar nuevos genes. Sin embargo, la diversidad no es tan alta, como para retrasar la convergencia del algoritmo en una solución óptima. Por ultimo la *Figura: 12* muestra que el recorrido de este individuo es el mismo recorrido que el presentada por el individuo de la fase 1, cuyo recorrido en la *Figura: 2* es por esta razón que los individuos tienen exactamente la misma aptitud y por lo cual podemos quedarnos con uno u con otro individuo.

Figura 10. Evolución de la aptitud

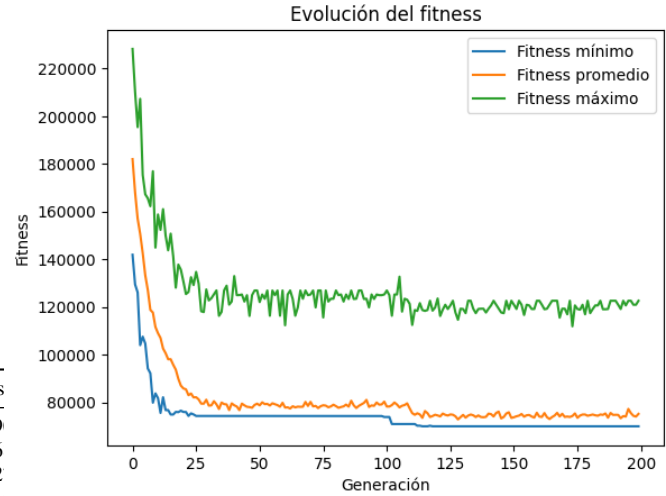


Figura 11. Evolución de la diversidad

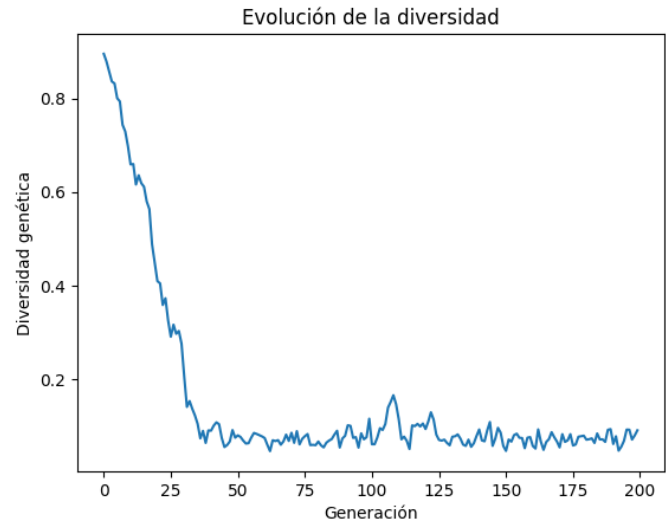
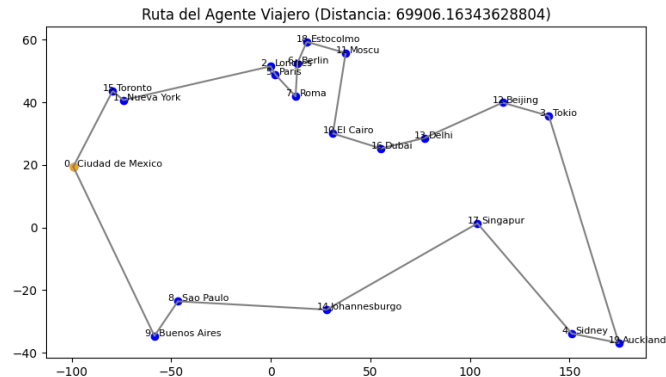


Figura 12. Recorrido del mejor individuo de la fase 3



En esta fase, pudimos observar como los parámetros del tamaño de la población y el de la tasa de mutación no parecen haber afectado de manera positiva nuestro algoritmo, ya que, independientemente de dichos parámetros, el algoritmo converge de manera eficaz. Por lo cual, podemos concluir

que donde realmente existe un factor de peso a la hora de obtener un resultado positivo es en el tipo de cruce y el tipo de mutación, siendo estos en donde realmente se decide la convergencia del algoritmo. Por lo cual, en esta ultima fase se ha demostrado que el **cruce de bordes** y la **mutación por intercambio** son los operadores con más peso, pero podemos decir que una tasa de mutación de 0,05 o 0,20 y una población entre 100 y 200 individuos son la configuración ideal para obtener individuos eficientes y que minimizan el recorrido del agente viajero.

II-D. Experimentación completa

Por ultimo y con curiosidad por saber como es que la experimentación en fases afectaba al desempeño del algoritmo, se decidió realizar la experimentación completa con todos los operadores presentados en la *Tabla: I*. Con dicha tabla en la que definimos todos los parámetros a experimentar, tenemos:

- **Cruza: 4 variantes (PMX, Bordes, Orden, Ciclos)**
- **Mutación: 4 variantes (Intercambio, Inserción, Desordenada, Inversión)**
- **Población: 3 variantes (50, 100, 200)**
- **Tasa de mutación: 3 variantes (5 %, 10 %, 20 %)**
- **Elitismo: 2 variantes (True, False)**

Estos parámetros nos dan un total de configuraciones de: $4 \times 4 \times 3 \times 3 \times 2 = 288$ **configuraciones**. Que se ejecutarán de manera independiente 30 veces; eso nos da un total de $288 \times 30 = 8640$ ejecuciones de nuestro algoritmo. Por lo cual, después de dejar ejecutando dichos experimentos, se obtuvo la *Tabla: V*. En dicha tabla se muestran los resultados de las configuraciones probadas; estos resultados nos servirán para analizar que tanto afecto haber realizado la experimentación en fases.

Tabla V
RESULTADOS DE TODAS LAS COMBINACIONES

Cruza	Mutacion	Mutacion	Poblacion	Elitismo	Fitness	Diversidad
Bordes	Desordenada	0.20	200	No	69906.16	0.1167
Orden	Desordenada	0.20	50	No	70805.84	0.153421
Bordes	Intercambio	0.10	100	No	70905.84	0.034211
PMX	Inversion	0.10	50	Si	70929.37	0.018947
PMX	Insercion	0.10	100	No	71021.63	0.018947

Como podemos observar, la configuración que está en primer lugar tiene la misma aptitud que las dos configuraciones vistas anteriormente, por lo cual podemos decir que el recorrido no depende únicamente de una configuración. Es interesante observar como, en este caso, existe una mayor variedad de configuraciones, métodos que quizás en las primeras fases no pudieron brillar. Por ejemplo, métodos de cruce como **PMX** o **Orden** que en la fase 1 no les fue tan bien, en esta experimentación con una configuración adecuada pueden generar soluciones optimas; pasa lo mismo con los métodos de mutación como **desordenada**, **inversión** o **inserción**, así como tamaños de población más pequeños.

Podemos observar nuevamente en la *Figura: 15* que el recorrido es el mismo que en las mostradas en 12, 2 y lo único que cambia es la evolución de su aptitud y la evolución de la

diversidad. Las *Figuras: 13, 14* muestran dichas evoluciones. La aptitud en esta configuración es mayormente variante en la aptitud máxima, ya que la aptitud media y la aptitud mínima se mantienen estables durante toda la evolución del algoritmo. La evolución de la diversidad también se mantiene estable, pero con algunos picos debido a que no se cuenta con elitismo; Sin embargo, esto no afecto a la evolución ya que se presenta un buen individuo.

Figura 13. Evolución de la aptitud

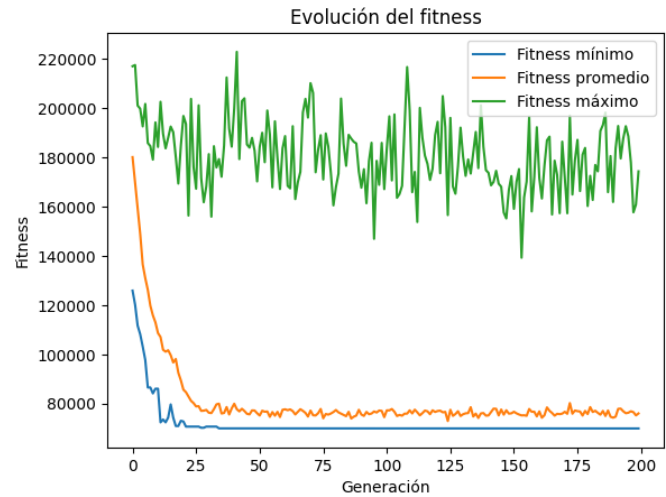


Figura 14. Evolución de la diversidad

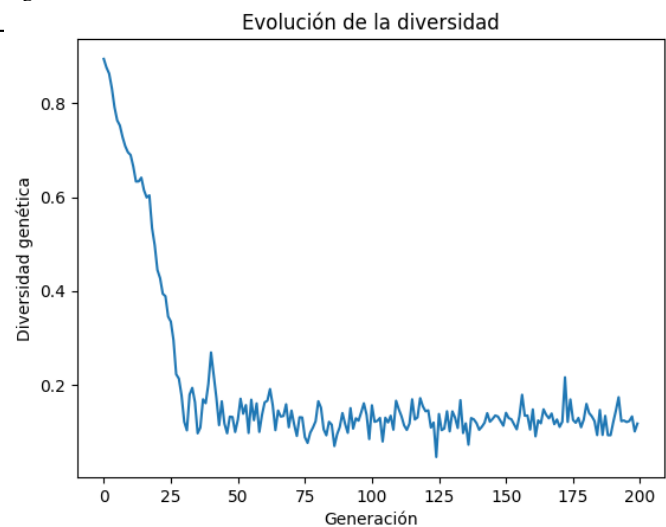
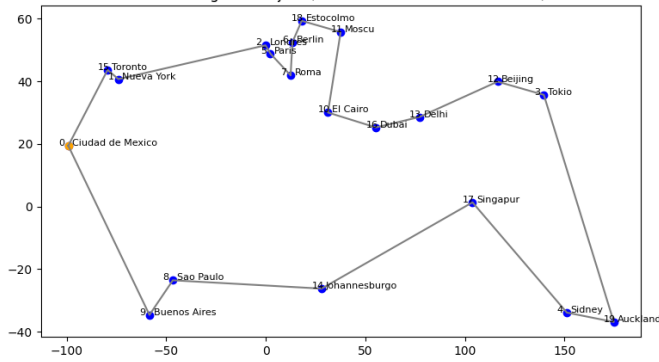


Figura 15. Recorrido del mejor individuo de todas las combinaciones
Ruta del Agente Viajero (Distancia: 69906.16343628804)



Este hallazgo es importante, ya que nos muestra que la experimentación por fases puede estar en cierto grado sesgada y que a veces puede ser mejor llevar a cabo la experimentación completa, ya que de esta manera se pueden encontrar configuraciones que no se probaron debido a la naturaleza de las fases y que igualmente funcionaban. Sin embargo, dado que en algunas ocasiones el espacio combinatorio puede ser demasiado grande, se deben optar por técnicas de experimentación que ayuden a evitar estos sesgos sin probar las millones de combinaciones.

III. CONCLUSIONES

Se implementó de manera correcta el algoritmo genético para resolver el Problema del Agente Viajero (TSP), logrando encontrar rutas altamente eficientes entre las 20 ciudades seleccionadas con la restricción de la ciudad inicial (Ciudad de México). A través de las diversas fases de experimentación, se obtuvieron algunas observaciones clave.

Se determinó que los operadores de cruce y mutación son el núcleo del desempeño del algoritmo. Mientras que el operador de cruce (especialmente el de Bordes) fue fundamental para realizar la explotación de las mejores aristas encontradas por los padres, la mutación (como la de intercambio) permitió la exploración del espacio de búsqueda, evitando que el algoritmo quedara atrapado en mínimos locales.

Posteriormente contrario a la hipótesis inicial, el elitismo, el tamaño de la población y la tasa de mutación no mostraron un impacto determinante de forma aislada. Esto sugiere que el AG para este tamaño de problema (20 ciudades) es robusto y que existe una interacción sinérgica entre parámetros; es decir, una tasa de mutación alta puede ser compensada por un elitismo fuerte, manteniendo la estabilidad de la solución óptima sin perder la diversidad genética necesaria. El seguimiento de la métrica de diversidad fue importante. Se observó que, a medida que las generaciones avanzaban, la diversidad disminuía conforme la población convergía hacia una zona común. El análisis de las gráficas de la aptitud (mínima, máxima y promedio) confirmó que el algoritmo no solo encontró una solución óptima individual, sino que logró elevar la calidad general de toda la población.

Uno de los hallazgos mas importantes es que se demostró que, a pesar de los buenos resultados obtenidos durante la experimentación por etapas, esta puede estar sesgada. Al

realizar el análisis de todas las combinaciones posibles, se revelaron configuraciones óptimas que no eran evidentes en las pruebas iniciales. Esto resalta la importancia de realizar búsquedas de hiperparámetros más exhaustivas en problemas de mayor complejidad. Dado que el TSP es un problema no tan complicado de resolver, la eficiencia del AG para encontrar una solución óptima valida la utilidad de la computación evolutiva.

En conclusión, la práctica permitió no solo resolver un problema clásico de optimización combinatoria, sino también comprender la sensibilidad de los algoritmos heurísticos ante su configuración. El éxito del algoritmo no residió en encontrar un unico parámetro, sino en el equilibrio entre preservar las buenas soluciones y permitir la aparición de nuevas variantes, garantizando así una búsqueda eficiente en un espacio de soluciones muy grande.

APÉNDICE A CÓDIGO FUENTE

```
1 import numpy as np
2 import random
3 import matplotlib.pyplot as plt
4 from geopy.distance import geodesic
5 import copy
6 import itertools
7 import pandas as pd
8
9 ciudades = [
10 {"nombre": "Ciudad de Mexico", "coords":
11     ↳ (19.4326, -99.1332)},
12 {"nombre": "Nueva York", "coords": (40.7128,
13     ↳ -74.0060)},
14 {"nombre": "Londres", "coords": (51.5074,
15     ↳ -0.1278)},
16 {"nombre": "Tokio", "coords": (35.6762,
17     ↳ 139.6503)},
18 {"nombre": "Sidney", "coords": (-33.8688,
19     ↳ 151.2093)},
20 {"nombre": "Paris", "coords": (48.8566,
21     ↳ 2.3522)},
22 {"nombre": "Berlin", "coords": (52.5200,
23     ↳ 13.4050)},
24 {"nombre": "Roma", "coords": (41.9028,
25     ↳ 12.4964)},
26 {"nombre": "Sao Paulo", "coords": (-23.5505,
27     ↳ -46.6333)},
28 {"nombre": "Buenos Aires", "coords":
29     ↳ (-34.6037, -58.3816)},
30 {"nombre": "El Cairo", "coords": (30.0444,
31     ↳ 31.2357)},
32 {"nombre": "Moscu", "coords": (55.7558,
33     ↳ 37.6173)},
34 {"nombre": "Beijing", "coords": (39.9042,
35     ↳ 116.4074)},
36 {"nombre": "Delhi", "coords": (28.6139,
37     ↳ 77.2090)},
```

```

24 {"nombre": "Johannesburgo", "coords":
    ↪ (-26.2041, 28.0473)},
25 {"nombre": "Toronto", "coords": (43.6532,
    ↪ -79.3832)},
26 {"nombre": "Dubai", "coords": (25.2769,
    ↪ 55.2962)},
27 {"nombre": "Singapur", "coords": (1.3521,
    ↪ 103.8198)},
28 {"nombre": "Estocolmo", "coords": (59.3293,
    ↪ 18.0686)},
29 {"nombre": "Auckland", "coords": (-36.8485,
    ↪ 174.7633)}
30 ]
31
32 def generar_poblacion(tam_poblacion):
33     poblacion = []
34
35     for _ in range(tam_poblacion):
36         resto = np.random.permutation(range(1,
37             ↪ 19))
38         individuo = np.concatenate(([0],
39             ↪ resto))
40         poblacion.append(individuo)
41
42     return np.array(poblacion)
43
44 def calcular_matriz_distancias(ciudades):
45     n = len(ciudades)
46     matriz = np.zeros((n, n))
47     for i in range(n):
48         for j in range(i + 1, n):
49             dist =
50                 ↪ geodesic(ciudades[i]["coords"],
51                 ↪ ciudades[j]["coords"]).km
52             matriz[i][j] = matriz[j][i] = dist
53
54     return matriz
55
56 def fitness_tsp(poblacion, matriz_distancias):
57     poblacion = np.array(poblacion)
58
59     distancias = np.sum(
60         matriz_distancias[poblacion[:, :-1],
61             ↪ poblacion[:, 1:]],
62         axis=1
63     )
64
65     regreso = matriz_distancias[poblacion[:,
66         ↪ -1], poblacion[:, 0]]
67
68     return distancias + regreso
69
70 def seleccionar_padres(poblacion, fitness,
71     ↪ n_padres=2, n_torneo=5):
72
73     indices_torneo =
74         ↪ np.random.choice(len(poblacion),
75         ↪ size=n_torneo, replace=False)
76
77     candidatos = []
78     for gen in indices_torneo:
79         candidatos.append((poblacion[gen],
80             ↪ fitness[gen]))
81
82     candidatos_ordenados = sorted(candidatos,
83         ↪ key=lambda x: x[1])
84
85     ganadores = [c[0] for c in
86         ↪ candidatos_ordenados[:n_padres]]
87
88     return ganadores
89
90 def mutacion_intercambio(individuo,
91     ↪ tasa_mutacion):
92     if random.random() < tasa_mutacion:
93         gen_1, gen_2 = random.sample(range(1,
94             ↪ len(individuo)), 2)
95         individuo[gen_1], individuo[gen_2] =
96             ↪ individuo[gen_2], individuo[gen_1]
97
98     return individuo
99
100 def mutacion_insercion(individuo,
101     ↪ tasa_mutacion):
102     if random.random() < tasa_mutacion:
103         gen_origen = random.randint(1,
104             ↪ len(individuo) - 1)
105         gen = individuo.pop(gen_origen)
106
107         gen_destino = random.randint(1,
108             ↪ len(individuo) - 1)
109         individuo.insert(gen_destino, gen)
110
111     return individuo
112
113 def mutacion_desordenada(individuo,
114     ↪ tasa_mutacion):
115     if random.random() < tasa_mutacion:
116         size = len(individuo)
117         gen_1, gen_2 =
118             ↪ sorted(random.sample(range(1,
119             ↪ size), 2))
120
121         subsegmento = individuo[gen_1:gen_2+1]
122         random.shuffle(subsegmento)
123
124         individuo[gen_1:gen_2+1] = subsegmento
125
126     return individuo
127
128 def mutacion_inversion(individuo,
129     ↪ tasa_mutacion):
130     if random.random() < tasa_mutacion:
131         size = len(individuo)

```



```

112     gen_1, gen_2 =
113         ↪ sorted(random.sample(range(1,
114             ↪ size), 2))
115
116     individuo[gen_1:gen_2+1] =
117         ↪ individuo[gen_1:gen_2+1][::-1]
118
119     return individuo
120
121 def cruce_pmx(padre1, padre2):
122     hijo1 = padre1.copy()
123     hijo2 = padre2.copy()
124
125     n = len(padre1)
126     punto1 = np.random.randint(1, n - 1)
127     punto2 = np.random.randint(punto1 + 1, n)
128
129     segmento1 = padre1[punto1:punto2]
130     segmento2 = padre2[punto1:punto2]
131
132     hijo1[punto1:punto2] = segmento2
133     hijo2[punto1:punto2] = segmento1
134
135     def resolver_duplicados(hijo,
136         segmento_propio,
137         segmento_ajeno,
138         p1, p2):
139         for i in np.concatenate([np.arange(1,
140             ↪ p1),
141             ↪ np.arange(p2, n)]):
142             candidato = hijo[i]
143             while candidato in segmento_ajeno:
144                 idx_en_ajeno =
145                     np.where(segmento_ajeno ==
146                         ↪ candidato)[0][0]
147                 candidato =
148                     ↪ segmento_propio[idx_en_ajeno]
149             hijo[i] = candidato
150     return hijo
151
152 def cruce_orden(p1, p2):
153     tamano = len(p1)
154     h1, h2 = [None]*tamano, [None]*tamano
155
156     h1[0], h2[0] = 0, 0
157
158     cruce_1, cruce_2 =
159         ↪ sorted(random.sample(range(1,
160             ↪ tamano), 2))
161
162     h1[cruce_1:cruce_2+1] =
163         ↪ p1[cruce_1:cruce_2+1]
164     h2[cruce_1:cruce_2+1] =
165         ↪ p2[cruce_1:cruce_2+1]
166
167     def rellenar_orden(hijo, padre, corte2):
168         size = len(padre)
169         pos_hijo = (corte2 + 1) % size
170         pos_padre = (corte2 + 1) % size
171
172         while None in hijo:
173             if pos_hijo == 0:
174                 pos_hijo = 1
175                 continue
176
177             ciudad_candidata = padre[pos_padre
178                 ↪ % size]
179             if ciudad_candidata not in hijo:
180                 hijo[pos_hijo] =
181                     ↪ ciudad_candidata
182                 pos_hijo = (pos_hijo + 1) %
183                     ↪ size
184                 pos_padre = (pos_padre + 1) % size
185
186     rellenar_orden(h1, p2, cruce_2)
187     rellenar_orden(h2, p1, cruce_2)
188     return h1, h2
189
190 def cruce_ciclos(p1, p2):
191     tamano = len(p1)
192     h1, h2 = [None]*tamano, [None]*tamano
193
194     h1[0], h2[0] = 0, 0
195
196     def resolver_ciclo(hijo, p_donante,
197         ↪ p_otro):
198         try:
199             inicio = next(i for i, v in
200                 ↪ enumerate(hijo) if v is None)
201         except StopIteration:
202             return
203
204         current = inicio
205         while hijo[current] is None:
206             hijo[current] = p_donante[current]
207             valor_buscado = p_otro[current]
208             current = list(p_donante)
209                 .index(valor_buscado)
210
211     turno_p1 = True
212     while None in h1:
213         if turno_p1:
214             resolver_ciclo(h1, p1, p2)
215             resolver_ciclo(h2, p2, p1)
216         else:
217             resolver_ciclo(h1, p2, p1)
218             resolver_ciclo(h2, p1, p2)

```

```

211     turno_p1 = not turno_p1
212
213     return h1, h2
214
215 def cruce_bordes(p1, p2):
216     size = len(p1)
217
218     adj = {i: set() for i in p1}
219     for p in [p1, p2]:
220         for i in range(size):
221             v1 = p[(i - 1) % size]
222             v2 = p[(i + 1) % size]
223             adj[p[i]].add(v1)
224             adj[p[i]].add(v2)
225
226     def generar_hijo(adj_copy):
227         hijo = []
228         current = 0
229
230         while len(hijo) < size:
231             hijo.append(current)
232
233             for v in adj_copy:
234                 adj_copy[v].discard(current)
235
236             if len(hijo) == size:
237                 break
238
239             vecinos = list(adj_copy[current])
240
241             if vecinos:
242                 min_grado =
243                     ↪ min(len(adj_copy[v]) for v
244                     ↪ in vecinos)
245                 candidatos = [v for v in
246                     ↪ vecinos if
247                     ↪ len(adj_copy[v]) ==
248                     ↪ min_grado]
249                 current =
250                     ↪ random.choice(candidatos)
251                     ↪ # clave
252             else:
253                 restantes = [n for n in
254                     ↪ adj_copy.keys() if n not
255                     ↪ in hijo]
256                 current =
257                     ↪ random.choice(restantes)
258
259         return hijo
260
261     h1 = generar_hijo(copy.deepcopy(adj))
262     h2 = generar_hijo(copy.deepcopy(adj))
263
264     return h1, h2
265
266 def seleccion_supervivientes(poblacion_actual,
267     ↪ hijos, fitness_actual, elitismo=True):
268     tam_poblacion = len(poblacion_actual)

```

```

258     hijos = np.array(hijos)
259
260     if not elitismo:
261         return hijos[:tam_poblacion]
262
263     n_elite = max(1, int(tam_poblacion *
264         ↪ 0.05))
265
266     idx_elite =
267         ↪ np.argpartition(fitness_actual,
268         ↪ n_elite)[:n_elite]
269     elites = poblacion_actual[idx_elite]
270
271     n_hijos_necesarios = tam_poblacion -
272         ↪ n_elite
273     idx = np.random.choice(len(hijos),
274         ↪ n_hijos_necesarios, replace=False)
275     hijos_seleccionados = hijos[idx]
276
277     nueva_poblacion = np.vstack((elites,
278         ↪ hijos_seleccionados))
279
280     return nueva_poblacion
281
282 def obtener_aristas(ruta):
283     aristas = set()
284     n = len(ruta)
285     for i in range(n):
286         u = ruta[i]
287         v = ruta[(i + 1) % n]
288         if u < v:
289             aristas.add((u, v))
290         else:
291             aristas.add((v, u))
292     return aristas
293
294 def calcular_diversidad(poblacion,
295     ↪ muestras=200):
296     n_pob = len(poblacion)
297     n_ciudades = len(poblacion[0])
298
299     aristas_pob = [obtener_aristas(ind) for
300         ↪ ind in poblacion]
301
302     distancias = []
303
304     for _ in range(muestras):
305         i, j = random.sample(range(n_pob), 2)
306         comunes = len(
307             ↪ aristas_pob[i]
308             ↪ .intersection(aristas_pob[j]))
309         dist_ij = 1 - (comunes / n_ciudades)
310         distancias.append(dist_ij)
311
312     return np.mean(distancias)

```

```

308 def algoritmo_genetico(metodo_cruza,
309     ↪ metodo_mutacion, con_elitismo,
310     ↪ tasa_mutacion, tamano_poblacion,
311     ↪ matriz_distancia, max_generaciones=30):
312     historial = {'min': [], 'max': [], 'mean':
313         ↪ [], 'div': []}
314
315     poblacion =
316     ↪ generar_poblacion(tamano_poblacion)
317
318     for generacion in range(max_generaciones):
319         # 1. Evaluación
320
321         aptitudes = fitness_tsp(poblacion,
322             ↪ matriz_distancia)
323
324         # 2. Registro de estadísticas
325         historial['min'].
326             .append(np.min(aptitudes))
327         historial['max'].
328             .append(np.max(aptitudes))
329         historial['mean'].
330             .append(np.mean(aptitudes))
331         historial['div'].
332             .append
333             (calcular_diversidad(poblacion))
334
335         # 3. Construcción de la nueva
336         ↪ generación
337         hijos = []
338
339         # Generamos suficientes hijos para
340         ↪ llenar la población
341         for _ in range(tamano_poblacion // 2):
342             padres =
343             ↪ seleccionar_padres(poblacion,
344             ↪ aptitudes)
345
346             h1, h2 =
347             ↪ metodo_cruza(list(padres[0]),
348             ↪ list(padres[1]))
349
350             h1 = list(h1)
351             h2 = list(h2)
352
353             h1 = metodo_mutacion(h1,
354                 ↪ tasa_mutacion)
355             h2 = metodo_mutacion(h2,
356                 ↪ tasa_mutacion)
357
358             h1 = np.array(h1)
359             h2 = np.array(h2)
360
361             hijos.extend([h1, h2])
362
363         poblacion =
364         ↪ seleccion_supervivientes(poblacion,
365         ↪ hijos, aptitudes, con_elitismo)
366
367         aptitudes_finales = fitness_tsp(poblacion,
368             ↪ matriz_distancia)
369
370         mejor_idx = np.argmin(aptitudes_finales)
371         mejor_individuo = poblacion[mejor_idx]
372
373         print(f"Mejor Distancia Final:
374             ↪ {aptitudes_finales[mejor_idx]} km")
375
376         return mejor_individuo, historial
377
378     configuraciones = {
379         'cruza': [cruce_pmx, cruce_bordes,
380             ↪ cruce_ciclos, cruce_orden],
381         'mutacion': [mutacion_intercambio],
382         'elitismo': [True, False],
383         'tasa_mutacion': [0.05],
384         'tam_poblacion': [100]
385     }
386
387     resultados_totales = []
388
389     combinaciones = list(itertools.product(
390         configuraciones['cruza'],
391         configuraciones['mutacion'],
392         configuraciones['elitismo'],
393         configuraciones['tasa_mutacion'],
394         configuraciones['tam_poblacion']
395     ))
396
397     print(f"Iniciando experimentos. Total
398         ↪ configuraciones: {len(combinaciones)}")
399
400     N_EJECUCIONES = 30
401     matriz_distancia =
402     ↪ calcular_matriz_distancias(ciudades)
403
404     for cruza, mutacion, elitismo, tasa_mutacion,
405     ↪ tam_poblacion in combinaciones:
406
407         nombre_config = f"{cruza.__name__} |
408             ↪ {mutacion.__name__} |
409             ↪ mut:{tasa_mutacion} |
410             ↪ pob:{tam_poblacion} | elit:{elitismo}"
411         print(f"\nEjecutando: {nombre_config}")
412
413         for ejecucion in range(N_EJECUCIONES):
414
415             mejor_ind, historial =
416             ↪ algoritmo_genetico(
417             ↪ cruza, mutacion, elitismo,
418             ↪ tasa_mutacion, tam_poblacion,
419             ↪ matriz_distancia,
420             ↪ max_generaciones=200
421             )
422
423             mejor_fitness = historial['min'][-1]

```

```

395
396     resultados_totales.append({
397         'Configuracion': nombre_config,
398         'Cruza': cruza.__name__,
399         'Mutacion': mutacion.__name__,
400         'Tasa_mutacion': tasa_mutacion,
401         'Tam_poblacion': tam_poblacion,
402         'Ejecución No.': ejecucion,
403         'Elitismo': elitismo,
404         'Mejor_Fitness': mejor_fitness,
405         'Diversidad_Final':
406             ↪ historial['div'][-1],
407     })
408
409 # DataFrame
410 df_resultados =
411     ↪ pd.DataFrame(resultados_totales)
412
413 # Guardar CSV
414 df_resultados.to_csv('resultados_tsp.csv',
415     ↪ index=False)
416
417 print("Experimentos completados")

```
